

Celebal Technologies Summer Internship Program 2026

Week 6 : Spark Assignment

Submitted By: Manjit Bajaj

Domain: Data Engineering

StudentId: CT_CSI_DE_1160

Objective: Understand Spark architecture and perform efficient data processing using transformations, filtering, schema handling, and optimized file formats.

Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Ans: 1. Driver

- The Driver is the main program that controls the Spark application.
- It creates the SparkSession and converts the user code into execution tasks.
- It builds the DAG (Directed Acyclic Graph) and schedules tasks for execution.
- It also collects the final results from the executors.

2. Cluster Manager

- The Cluster Manager manages the resources available in the cluster.
- It allocates CPU cores and memory to Spark applications.
- It launches executors on worker nodes.
- Examples include Standalone Cluster Manager, YARN, Kubernetes, and Apache Mesos.

3. Executors

- Executors are worker processes that execute the tasks assigned by the Driver.
- They perform computations on the data.
- They store intermediate data in memory.
- After execution, they return the results to the Driver.

Q2. How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Ans:

- Spark does not execute transformations immediately.
- Instead, it records all transformations in a Directed Acyclic Graph (DAG).
- Execution starts only when an action such as show() or count() is called.
- Spark combines multiple transformations into a single optimized execution plan.
- This reduces unnecessary data movement and minimizes disk I/O.
- As a result, Spark processes large datasets faster and more efficiently.

Q3. Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

Ans:

```

from google.colab import files

uploaded = files.upload()

Choose Files Superstore.csv
Superstore.csv(text/csv) - 2287806 bytes, last modified: 6/27/2026 - 100% done
Saving Superstore.csv to Superstore.csv

df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv("Superstore.csv")

df.printSchema()
df.show(5)

root
|-- Row ID: integer (nullable = true)
|-- Order ID: string (nullable = true)
|-- Order Date: string (nullable = true)

```

Q4. What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

Ans:

CSV	Parquet
Row-based storage	Columnar storage
Larger file size	Smaller compressed files
Slower query performance	Faster query performance
Reads the entire row	Reads only required columns
Suitable for data exchange	Best for analytics and big data

Why it matters

- Parquet stores data column-wise, allowing Spark to read only the required columns.
- It supports compression and Predicate Pushdown, reducing disk I/O.
- Therefore, Parquet provides faster query execution than CSV.

Q5. Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

Ans:

In the superstore dataset, there was no column name with price so I have used sales instead of it. Category Electronics does not exist.

```
df.select("Category").distinct().show()
```

```
+-----+
|      Category|
+-----+
|Office Supplies|
|      Furniture|
|      Technology|
+-----+
```

```
from pyspark.sql.functions import col

df.select("product_id","sales") \
    .filter(col("category") == "Electronics") \
    .show()
```

```
+-----+-----+
|product_id|sales|
+-----+-----+
+-----+-----+
```

Q6. Write the code to revise a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

Ans:

```
# rename columns
df = (df.withColumnRenamed("Row ID", "row_id")
      .withColumnRenamed("Order ID", "order_id")
      .withColumnRenamed("Customer ID", "customer_id")
      .withColumnRenamed("Customer Name", "customer_name")
      .withColumnRenamed("Segment", "segment")
      .withColumnRenamed("Country", "country")
      .withColumnRenamed("City", "city")
      .withColumnRenamed("State", "state")
      .withColumnRenamed("Postal Code", "postal_code")
      .withColumnRenamed("Region", "region")
      .withColumnRenamed("Product ID", "product_id")
      .withColumnRenamed("Category", "category")
      .withColumnRenamed("Sub-Category", "sub_category")
      .withColumnRenamed("Product Name", "product_name")
      .withColumnRenamed("Sales", "sales")
      .withColumnRenamed("Quantity", "quantity")
      .withColumnRenamed("Discount", "discount")
      .withColumnRenamed("Profit", "profit")
      .withColumnRenamed("Order Date", "order_date")
      .withColumnRenamed("Ship Date", "ship_date")
      .withColumnRenamed("Ship Mode", "ship_mode"))

print(df.columns)
```

```
... ['row_id', 'order_id', 'order_date', 'ship_date', 'ship_mode', 'customer_id', 'customer_name', 'segment', 'country', 'city', 'state', 'postal_code', 'region', 'product_id',
```

```
# Data type conversion
from pyspark.sql.functions import col

df = df.withColumn(
    "discount",
    col("discount").cast("double")
)
```

Since, Price columns does not exist.

Q7. How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Ans:

- Spark records every transformation in a Lineage Graph (DAG).
- The DAG stores the sequence of operations performed on the data.
- If an executor or worker node fails, Spark does not reload the entire dataset.
- Instead, it recomputes only the lost partitions using the lineage information.
- This provides efficient fault tolerance without requiring data replication.

Q8. Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

Ans: from pyspark.sql.functions import col
df_orders.filter(
 (col("status") == "Completed") &
 (col("amount") > 1000)
).show()

Q9. Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Ans:

- Predicate Pushdown is an optimization technique used by Spark with Parquet files.
- Spark pushes filter conditions to the storage layer before reading the data.
- Only the rows matching the filter are loaded into memory.
- This reduces disk I/O, memory usage, and query execution time.
- As a result, Spark processes data more efficiently.

Q10. Write a code snippet to add a new column final_price which is the base_price multiplied by 1.18 (18% tax).

Ans:

```
# Creating new columns
from pyspark.sql.functions import round

df = df.withColumn(
    "final_sales",
    round(col("sales") * 1.18, 2)
)

df = df.withColumn(
    "discount_amount",
    round(col("sales") * col("discount"), 2)
)

df.select("sales", "final_sales", "discount_amount", "discount").show(10)
```

sales	final_sales	discount_amount	discount
261.96	309.11	0.0	0.0
731.94	863.69	0.0	0.0
14.62	17.25	0.0	0.0
1053.5376	1243.17	123.84	0.15

Q11. What is the difference between Transformations and Actions? Provide two examples of each.

Ans:

Transformations

- Transformations create a new DataFrame without executing immediately.
- They are evaluated lazily.

Examples

- filter()
- select()

Actions

- Actions trigger the execution of all pending transformations.
- They return results or write data.

Examples

- show()
- count()

Q12. Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

Ans:

Code

```
from pyspark.sql.functions import col
```

```
spark.read.parquet("path/to/input") \
    .filter(col("user_id").isNotNull()) \
    .write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("path/to/output")
```

Q13. In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Ans:

Client Mode	Cluster Mode
Driver runs on the client machine.	Driver runs inside the cluster.
Suitable for development and testing.	Suitable for production environments.
Client machine must remain active.	Application continues even if the client disconnects.
Easier for debugging.	Better reliability and scalability.

Q14. Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

Ans:

```
from pyspark.sql.functions import col

df.filter(
    (col("region") == "North")
).show()
```

row_id	order_id	order_date	ship_date	ship_mode	customer_id	customer_name	segment	country	city	state	postal_code	region	product_id
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-10001798
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-10000454
3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-10000240
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577
5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	OFF-ST-10000760

Code

```
from pyspark.sql.functions import col
```

```
df.filter(
    (col("region") == "North") |
    (col("priority") == "High")
).show()
```

Q15. When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

Ans:

```
df.show(5)
```

row_id	order_id	order_date	ship_date	ship_mode	customer_id	customer_name	segment	country	city	state	postal_code	region	product_id
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-10001798
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-10000454
3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-10000240
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577
5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	OFF-ST-10000760

only showing top 5 rows

```
df.collect()
```

```
Row(row_id=764, order_id='CA-2014-162775', order_date='1/13/2014', ship_date='1/15/2014', ship_mode='Second Class', customer_id='CS-12250', customer_name='Chris Selesnick', segment='Corporate', country='United States', city='Bossier City', state='Louisiana', postal_code=71111, region='South', product_id='OFF-EN-10001990', category='Office Supplies', sub_category='Envelopes', product_name='Staple envelope', sales=11.36, quantity=2, discount=0.0, profit=5.3392, final_sales=13.4, discount_amount=0.0),
Row(row_id=765, order_id='CA-2014-162775', order_date='1/13/2014', ship_date='1/15/2014', ship_mode='Second Class', customer_id='CS-12250', customer_name='Chris Selesnick', segment='Corporate', country='United States', city='Bossier City', state='Louisiana', postal_code=71111, region='South', product_id='OFF-EN-10001532', category='Office Supplies', sub_category='Envelopes', product_name='Brown Kraft Recycled Envelopes', sales=50.94, quantity=3, discount=0.0, profit=25.47,
```

- .show(5) displays only the first five rows of the dataset.
- It avoids transferring the entire dataset to the Driver.
- .collect() loads all records into the Driver's memory.
- For very large datasets, .collect() may cause memory overflow or application failure.
- Therefore, .show(5) is a safer and more efficient choice for data exploration.

Conclusion: This assignment demonstrated a complete ETL pipeline in Apache Spark, including data loading, transformation, filtering, and saving using efficient DataFrame operations.